

02 — Dimension Reduction

From the course syllabus (Practicals 6–13), the only dimension reduction algorithm covered is **PCA**.

When does the exam ask for dimension reduction?

- Input has many features (e.g. 5) and you need to fit a 1D/2D model
- Features are highly correlated — PCA collapses them into one direction

The 5-step PCA recipe (memorize this order):

1. Normalize (mean=0, std=1)
2. Covariance matrix
3. Eigendecomposition
4. Sort eigenvalues descending
5. Project onto first k directions

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
```

Step 1 — Normalize the training data

Why? Features on different scales (e.g. $x_0 \sim 0-30$, $x_3 \sim 0-200$) would make PCA biased toward large-scale features. Normalization puts every feature on equal footing.

Rule: compute `mean` and `std` from training data only. Apply to test data later.

```
In [9]: # Assume X_train is shape (m, n) – m samples, n features
# (Pretend we loaded train.csv as in notebook 01)

# --- dummy data for this notebook to run standalone ---
np.random.seed(0)
X_train = np.column_stack([
    np.random.randn(50) * 10 + 15,
    np.random.randn(50) * 30 + 50,
    np.random.randn(50) * 20 + 35,
    np.random.randn(50) * 80 + 130,
    np.random.randn(50) * 50 + 80
]) # shape (50, 5)
# -----

mu_X    = X_train.mean(axis=0) # mean of each feature column → shape (n,)
sigma_X = X_train.std(axis=0)  # std of each feature column → shape (n,)
```

```
X_train_norm = (X_train - mu_X) / sigma_X    # shape (m, n)

print('mu_X:', mu_X)
print('After norm - means:', X_train_norm.mean(axis=0).round(10), '← should be ~0')
print('After norm - stds: ', X_train_norm.std(axis=0), '← should be ~1')
print('X_train_norm.shape', X_train_norm.shape)
```

```
mu_X: [ 16.40559272  49.37170276  40.1306968  122.59928811  73.76307584]
After norm - means: [-0.  0. -0.  0. -0.] ← should be ~0
After norm - stds:  [1.  1.  1.  1.  1.] ← should be ~1
X_train_norm.shape (50, 5)
```

Step 2 — Covariance matrix

```
Sigma = (1/m) * X_norm.T @ X_norm
```

Shape is (n, n). Each entry `Sigma[i, j]` = how features i and j vary together. Off-diagonal values near ± 1 mean the features are highly correlated. If all features are correlated, PCA will capture almost all variance in PC1 alone.

```
In [13]: m = X_train_norm.shape[0]
Sigma = (1/m) * X_train_norm.T @ X_train_norm    # shape (n, n)

print('Covariance matrix (n x n):')
print(Sigma)
# Diagonal = variance of each normalized feature (should be ~1)
# Off-diagonal = correlation between pairs of features
```

```
Covariance matrix (n x n):
[[ 1.          -0.05918059  0.06357795 -0.09718223  0.0979902 ]
 [-0.05918059  1.          -0.15720501  0.14995654 -0.14478921]
 [ 0.06357795 -0.15720501  1.          -0.18477785  0.08749489]
 [-0.09718223  0.14995654 -0.18477785  1.          -0.0060917 ]
 [ 0.0979902  -0.14478921  0.08749489 -0.0060917  1.          ]]
```

Step 3 — Eigendecomposition

`np.linalg.eig(Sigma)` returns:

- `eigenvalues` shape (n,) — how much variance each direction holds
- `eigenvectors` shape (n, n) — each **column** is one principal component direction

Add `.real` to drop tiny imaginary rounding errors that `eig` sometimes produces.

```
In [15]: eigenvalues, eigenvectors = np.linalg.eig(Sigma)

print('Eigenvalues (raw):', eigenvalues.round(4))
print('Eigenvectors shape:', eigenvectors.shape)    # (n, n) - each column = one PC
```

```

Eigenvalues (raw): [1.4332+0.j 1.016 +0.j 0.954 +0.j 0.7679+0.j 0.8289+0.j]
Eigenvectors shape: [[ 0.33438047+0.j -0.32594984+0.j -0.83977889+0.j -0.27365139+0.j
-0.04281056+0.j]
[-0.51729433+0.j 0.05407638+0.j -0.39616603+0.j 0.42087471+0.j
0.62880782+0.j]
[ 0.51314902+0.j 0.28491238+0.j 0.16418458+0.j -0.32902648+0.j
0.72130993+0.j]
[-0.4779909 +0.j -0.52782519+0.j 0.21095451+0.j -0.63568197+0.j
0.21055141+0.j]
[ 0.35888586+0.j -0.72873723+0.j 0.25761505+0.j 0.48541673+0.j
0.19531531+0.j]]

```

Step 4 — Sort eigenvalues descending (largest variance first)

`np.argsort` gives indices for ascending order. `[::-1]` reverses to descending. You must sort the **columns** of eigenvectors — each column is one direction.

```

In [16]: order      = np.argsort(eigenvalues)[::-1]      # indices: largest eigenvalue fir
eigenvalues = eigenvalues[order].real                  # sort eigenvalues, drop imagina
eigenvectors = eigenvectors[:, order].real              # sort COLUMNS (not rows!)

total_var = eigenvalues.sum()
for i, lam in enumerate(eigenvalues):
    pct = lam / total_var * 100
    print(f' PC{i+1}: eigenvalue = {lam:.4f}    ({pct:.1f}% of variance)')

PC1: eigenvalue = 1.4332    (28.7% of variance)
PC2: eigenvalue = 1.0160    (20.3% of variance)
PC3: eigenvalue = 0.9540    (19.1% of variance)
PC4: eigenvalue = 0.8289    (16.6% of variance)
PC5: eigenvalue = 0.7679    (15.4% of variance)

```

Step 5 — Project data onto top k components

For the exam, usually $k=1$ (reduce to 1D for regression). `U1` = first column of sorted eigenvectors, shape (n, 1). Projection = `X_norm @ U1`, shape (m, 1). Flatten to (m,).

```

In [17]: # --- Project to 1D (k = 1) ---
U1      = eigenvectors[:, :1]      # shape (n, 1) – first principal direct
x1_train = (X_train_norm @ U1).flatten() # (m, n) @ (n, 1) = (m, 1) → flatten → (

print('x1_train shape:', x1_train.shape) # (50,)
print(f'PC1 explains {eigenvalues[0]/total_var*100:.1f}% of variance')

x1_train shape: (50,)
PC1 explains 28.7% of variance

```

```
In [18]: # --- Project to 2D (k = 2) - if the exam asks for 2 components ---
U2      = eigenvectors[:, :2]          # shape (n, 2)
X_2d_train = X_train_norm @ U2        # shape (m, 2)

print('X_2d_train shape:', X_2d_train.shape)    # (50, 2)
var_explained = eigenvalues[:2].sum() / total_var * 100
print(f'PC1+PC2 explain {var_explained:.1f}% of variance')
```

X_2d_train shape: (50, 2)
PC1+PC2 explain 49.0% of variance

Full PCA as one function

Copy-paste this into the exam and call it.

```
In [29]: def PCA(X_train, k=1):
        """
        Reduce X_train from (m, n) to (m, k).
        Returns: x_reduced, mu_X, sigma_X, U
        Keep mu_X, sigma_X, U to apply to test data later!
        """
        mu_X = X_train.mean(axis=0)
        sigma_X = X_train.std(axis=0)
        X_norm = (X_train - mu_X) / sigma_X

        m = X_norm.shape[0]
        Sigma = (1/m) * X_norm.T @ X_norm

        eigenvalues, eigenvectors = np.linalg.eig(Sigma)
        order = np.argsort(eigenvalues)[::-1]
        eigenvalues = eigenvalues[order].real
        eigenvectors = eigenvectors[:, order].real

        U = eigenvectors[:, :k]          # (n, k)
        x_reduced = (X_norm @ U).flatten() if k == 1 else X_norm @ U    # (m,) or (m, k)

        pct = eigenvalues[:k].sum() / eigenvalues.sum() * 100
        print(f'Top {k} PC(s) explain {pct:.1f}% of variance')

        return x_reduced, mu_X, sigma_X, U

k = 1
x1_train, mu_X, sigma_X, U1 = PCA(X_train, k)
print('x1_train shape:', x1_train.shape)
```

Top 1 PC(s) explain 28.7% of variance
x1_train shape: (50,)

Apply PCA to test data

Use the **training** `mu_X`, `sigma_X`, and `U1` — never recompute from test.

```
In [30]: # Assume X_test is loaded (shape (m_test, n))
X_test = X_train[:10] # dummy: just first 10 rows

X_test_norm = (X_test - mu_X) / sigma_X # use TRAINING mu_X, sigma_X
x1_test      = (X_test_norm @ U1).flatten() if k == 1 else (X_test_norm @ U1) # use
print('x1_test shape:', x1_test.shape)
```

x1_test shape: (10,)

Summary

Step	Code	Output shape
Normalize	<code>(X - mu) / std</code>	<code>(m, n)</code>
Covariance	<code>(1/m) * X_norm.T @ X_norm</code>	<code>(n, n)</code>
Eigen	<code>np.linalg.eig(Sigma)</code>	eigenvalues <code>(n,)</code> , eigenvectors <code>(n,n)</code>
Sort	<code>eigenvectors[:, np.argsort(vals)[:-1]]</code>	<code>(n, n)</code>
Project 1D	<code>(X_norm @ U1).flatten()</code>	<code>(m,)</code>
Project kD	<code>X_norm @ U_k</code>	<code>(m, k)</code>

Common mistake: sorting `eigenvectors[order]` (sorts rows) instead of `eigenvectors[:, order]` (sorts columns). Always sort columns!